



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: JULIO – DICIEMBRE 2026



INFORME DE GUÍA PRÁCTICA

I. PORTADA

UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS, ELECTRÓNICA E INDUSTRIAL



Carrera de ingeniera: Software

Título: La historia y evolución de los lenguajes de programación

Unidad de Organización Curricular: Basica

Alumnos y participantes: Calderon Carvajal Fher Dorian

Ciclo Académico y Paralelo: Primer Semestre “B”

**Módulo y Docente: Algoritmos y lógica de programación -
Ing. Caiza Carizabuan José Ruben**

LUNES / 23 DE AGOSTO / 2026



II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Conocer los lenguajes de programación de la actualidad y sus antecesores

Específicos:

- Describir la evolución histórica de los lenguajes de programación, desde sus antecedentes hasta los lenguajes actuales.
- Clasificar los lenguajes de programación según su nivel de abstracción: alto nivel, nivel medio y bajo nivel.
- Comparar las características, formas de traducción y entornos de desarrollo de un lenguaje de nivel medio (C++) y uno de alto nivel (Java).

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 2

No presenciales: 0

2.4 Instrucciones

- El trabajo se desarrollará de forma individual
- Realice una consulta sobre LA HISTORIA Y EVOLUCIÓN DE LOS LENGUAJES DE PROGRAMACIÓN.
- Seleccionar una herramienta que permita realizar infografías.
- Realizar una infografía a manera de resumen de la lectura realizada.
- Subir el documento al aula virtual en el formato de Guías APE en PDF

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

- Inteligencia artificial
- TAC Inteligencia artificial
- TAC - Computador o laptop. - Plataforma educativa de la UTA. - Internet para consultas (fuentes confiables).

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☒ Plataformas educativas
- ☐ Simuladores y laboratorios virtuales
- ☒ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☐ Gamificación
- ☒ Inteligencia Artificial



Otros (Especifique): _____

2.6 Actividades por desarrollar

Desarrollar la guía APE en base al formato y las instrucciones brindadas: investigar la historia y evolución de los lenguajes de programación, clasificarlos por nivel de abstracción y elaborar una infografía que resuma lo investigado.

2.7 Resultados obtenidos

SW-AYLP-APE-01 · ALGORITMOS Y LÓGICA DE PROGRAMACIÓN

De la tarjeta perforada al *bytecode*

Historia y evolución de los lenguajes de programación

ERA 0

Antes de la máquina

1804 - 1936

1804

Telar de Jacquard
Tarjetas perforadas controlan el patrón de un tejido: la primera vez que una máquina sigue instrucciones externas.^[1]

1843

Ada Lovelace
Traduce las notas de Babbage sobre la máquina analítica y añade un método propio para calcular números de Bernoulli. Se la considera la primera programadora de la historia.^[1]

1936

Turing y Church
Formulan, cada uno por su lado, un modelo matemático de «máquina universal». Resultó que ambos son equivalentes.^[1]

ERA 1

Bajo nivel: hablarle a la máquina en su idioma

1941 - 1956

1941-48

Konrad Zuse — Z3, Z4 y Plankalkül
El Z3 es el primer ordenador programable que funcionó; Plankalkül, diseñado en 1948, se considera el primer lenguaje de programación, aunque nunca se llegó a construir.^[1,2]

1949

Lenguaje ensamblador (EDSAC)
Mnemónicos como ADD o STORE reemplazan al binario puro; un programa traductor —el ensamblador— los convierte a código máquina.^[1,2]



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: JULIO – DICIEMBRE 2026



ERA 2	Nace el alto nivel	1957 – 1964
		1957
	FORTRAN — John Backus, IBM El primer lenguaje de alto nivel de uso masivo con implementación real; pensado para cálculo científico. ^[1,2]	
		1958
	LISP — John McCarthy, MIT Basado en listas, orientado al procesamiento simbólico: la raíz de la investigación temprana en IA. ^[1,2]	
		1958-60
	ALGOL Un comité internacional busca un lenguaje independiente de la máquina. De él descenden, directa o indirectamente, Pascal, C, Ada y hasta Java. ^[1,2]	
		1960
	COBOL — comité CODASYL / Grace Hopper Sintaxis casi en inglés para que los programas de gestión fueran legibles y portátiles entre máquinas. ^[1,2]	
		1964
	BASIC — Kemeny y Kurtz, Dartmouth Pensado para que estudiantes sin formación técnica programaran; décadas después aún sobrevive en Visual Basic. ^[2]	
ERA 3	Nivel medio y el primer objeto	1962 – 1985
		1962
	Simula — Dahl y Nygaard Pensado para simular eventos discretos, termina siendo el primer lenguaje con clases y objetos. ^[1,2]	
		1970
	Pascal — Niklaus Wirth Hereder de ALGOL 60, diseñado específicamente para enseñar a programar con estructura. ^[1,2]	
		1972
	C — Dennis Ritchie, Bell Labs Nace de B para reescribir UNIX. Se le llama lenguaje de nivel medio : tan legible como uno de alto nivel, pero con acceso directo a la memoria y al sistema operativo. ^[1,2]	
		1972
	Prolog — Colmerauer y Kowalski En vez de dar órdenes paso a paso, se describen hechos y reglas; el intérprete deduce la respuesta. ^[1,2]	
		1980
	Smalltalk — Alan Kay, Xerox PARC Todo —números, ventanas, hasta el propio código— es un objeto que recibe mensajes. ^[1,2]	
		1985
	C++ — Bjarne Stroustrup Añade clases y objetos a C sin renunciar al control cercano al hardware que lo hizo popular. ^[1,2]	



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: JULIO – DICIEMBRE 2026



ERA 4	La era moderna	1991 – hoy
	Python — Guido van Rossum	1991
	Sintaxis pensada para leerse casi como pseudocódigo; hoy es multiparadigma e interpretado. ^[1]	
	Java — James Gosling, Sun Microsystems	1995
	Nació pensando en electrodomésticos inteligentes; terminó como el lenguaje de «escribelo una vez, ejecútalo en cualquier máquina» gracias a su máquina virtual. ^[1, 2]	
	JavaScript, PHP, Ruby	1995
	El mismo año, la web emergente se escribe sus propios lenguajes: cliente, servidor y propósito general. ^[1]	
	C# — Anders Hejlsberg, Microsoft	2000
	Lenguaje orientado a objetos diseñado junto con la plataforma .NET, pensado para competir con Java. ^[1]	
	Go, Swift, Kotlin, Rust...	hoy
	Cada uno ataca un problema que Java o C++ dejaron abierto: concurrencia, seguridad de memoria, apps móviles. ^[1]	

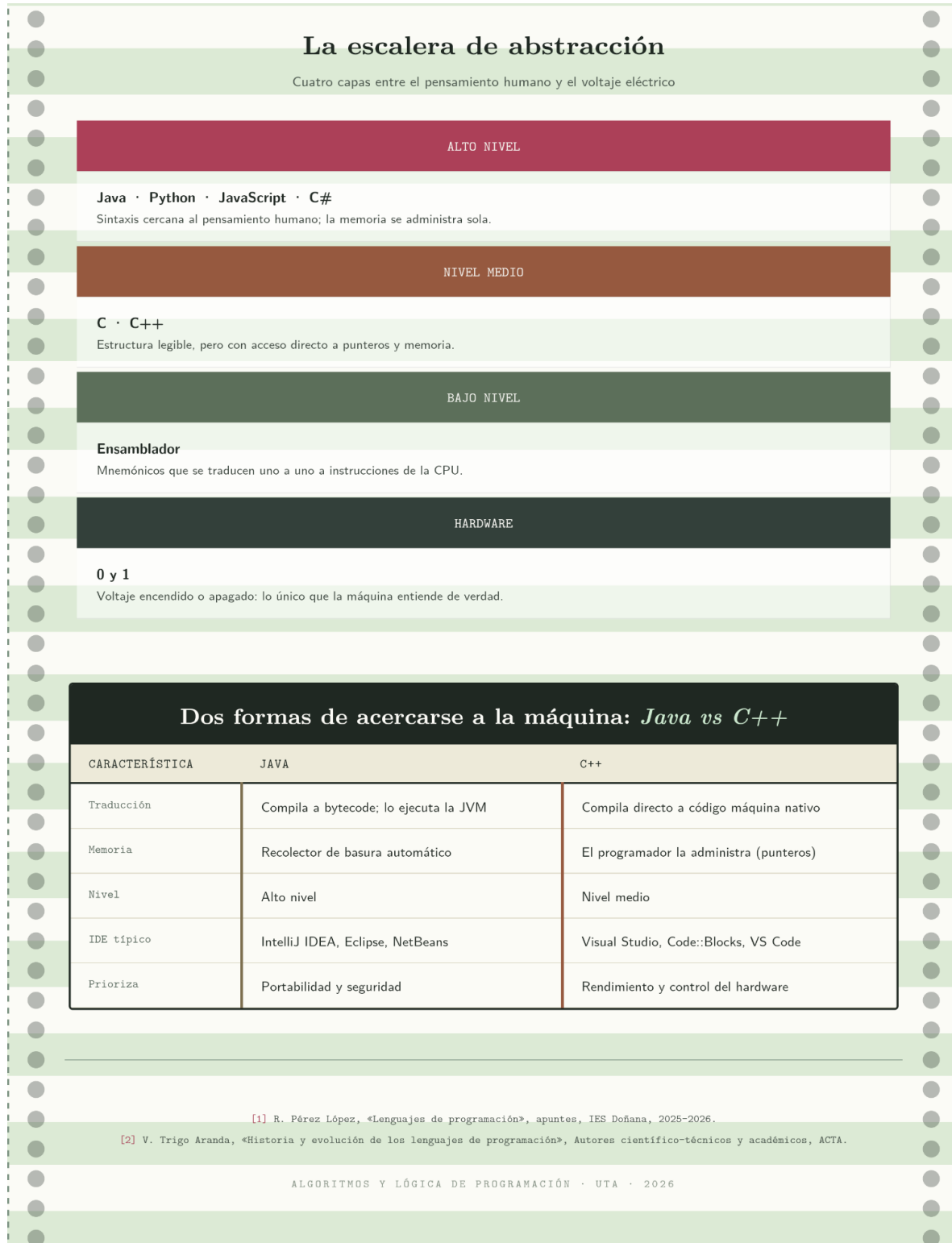


Figura 1. Línea de tiempo de la evolución de los lenguajes de programación y escalera de abstracción. Elaboración propia a partir de [1], [2].



2.8 Habilidades blandas empleadas en la práctica

- ☐ Liderazgo
- ☐ Trabajo en equipo
- ☐ Comunicación asertiva
- ☐ La empatía
- ☒ Pensamiento crítico
- ☒ Flexibilidad
- ☐ La resolución de conflictos
- ☒ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones

Recorrer la evolución de los lenguajes de programación deja claro que no existe un “mejor” lenguaje en términos absolutos: cada uno nació para resolver un problema concreto de su época, desde controlar un telar mecánico hasta portar una misma aplicación a miles de dispositivos distintos. La clasificación por nivel —bajo, medio y alto— no es solo una etiqueta académica: explica por qué C++ sigue siendo la opción cuando se necesita control fino sobre el hardware, mientras que Java o Python resultan más productivos cuando lo prioritario es la portabilidad y la velocidad de desarrollo. Conocer esta historia y esta clasificación permite elegir con criterio el lenguaje que mejor se adapte a las necesidades de un proyecto de software real, en lugar de aprender un lenguaje aislado sin entender qué lugar ocupa dentro del panorama general de la programación.

2.10 Recomendaciones

- Profundizar el estudio de C++ y Java en las siguientes semanas del curso, dado que ambos representan los niveles medio y alto de la escalera de abstracción revisada en esta guía.
- Instalar y familiarizarse con al menos un IDE por lenguaje (por ejemplo, Visual Studio Code para C++ y IntelliJ IDEA o Eclipse para Java) antes de las próximas prácticas de codificación.



- Consultar la documentación oficial de cada lenguaje como fuente primaria antes de recurrir a foros de comunidad, siguiendo el criterio de selección de fuentes revisado en clase.

2.11 Referencias bibliográficas

- [1] R. Pérez López, “Lenguajes de programación”, apuntes de clase, IES Doñana, curso 2025/2026. [En línea]. Disponible: <https://pro.iesdonana.org/apuntes/lenguajes-de-programacion-apuntes.pdf>
- [2] V. Trigo Aranda, “Historia y evolución de los lenguajes de programación”, Autores científico-técnicos y académicos, Asociación de Técnicos de Informática (ATI). [En línea]. Disponible: https://www.acta.es/medios/articulos/informatica_y_computacion/034083.pdf

2.12 Anexos

2.13 Informe

Antecedentes y evolución histórica

Antes de que existiera el primer ordenador electrónico ya existían máquinas capaces de seguir instrucciones. En 1804, el telar de Jacquard usaba tarjetas perforadas para controlar el patrón de un tejido: una máquina que “leía” instrucciones externas para producir un resultado distinto según los datos que recibía [1]. Décadas más tarde, cuando Charles Babbage diseñó la máquina analítica, Ada Lovelace tradujo las notas de Luigi Menabrea sobre ese diseño y añadió un método propio para calcular los números de Bernoulli; por ese trabajo se la considera la primera programadora de la historia [1]. En 1936, Alan Turing y Alonzo Church formularon, cada uno por su lado, un modelo matemático de lo que hoy llamamos una máquina universal; ambos modelos resultaron equivalentes y sentaron las bases teóricas de la computación moderna [1].

Los primeros ordenadores programables reales llegaron con el trabajo de Konrad Zuse: el Z3 (1941) y el Z4 (1945). En 1948, Zuse diseñó Plankalkül, considerado el primer lenguaje de programación de la historia, aunque nunca llegó a implementarse en su época [1, 2]. Hasta ese momento, programar significaba escribir directamente en código máquina. En 1949 apareció el lenguaje ensamblador, que



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: JULIO – DICIEMBRE 2026



sustituyó las cadenas de ceros y unos por mnemónicos como ADD o STORE, traducidos a código máquina mediante un programa ensamblador [2].

El primer lenguaje de alto nivel de uso masivo fue FORTRAN, desarrollado por John Backus en IBM y terminado en 1957 [1, 2]. Poco después llegaron LISP (1958), orientado al procesamiento simbólico y a la naciente inteligencia artificial, y la familia ALGOL (1958-1960), un intento de crear un lenguaje independiente de la máquina del que descienden, directa o indirectamente, Pascal, C, Ada e incluso Java [1, 2]. En 1960, un comité liderado por Grace Hopper desarrolló COBOL, pensado para aplicaciones de gestión con una sintaxis cercana al inglés [1, 2]. Cuatro años más tarde, en Dartmouth, John Kemeny y Thomas Kurtz crearon BASIC para que estudiantes sin formación técnica pudieran programar [2].

En 1962, los noruegos Ole-Johan Dahl y Kristen Nygaard crearon Simula, pensado originalmente para simular eventos discretos; se lo considera el primer lenguaje con clases y objetos [1]. En 1970, Niklaus Wirth presentó Pascal, heredero de ALGOL 60 y diseñado específicamente para enseñar a programar de forma estructurada [1, 2]. Dos años después, en los Laboratorios Bell, Dennis Ritchie creó C a partir de un lenguaje experimental llamado B; C se describe habitualmente como un lenguaje de **nivel medio**, porque combina una sintaxis tan legible como la de un lenguaje de alto nivel con acceso directo a la memoria y al sistema operativo [1]. Ese mismo año, Alain Colmerauer y Robert Kowalski propusieron Prolog, el primer lenguaje de programación lógica, en el que en lugar de indicar los pasos a seguir se describen hechos y reglas, y es el propio intérprete el que deduce la respuesta [1, 2].

La programación orientada a objetos maduró en la década de 1980. Alan Kay, en el Xerox PARC, desarrolló Smalltalk (1980), donde absolutamente todo incluidos los números es un objeto que recibe mensajes [1, 2]. En 1985, Bjarne Stroustrup extendió C con clases y objetos para crear C++, sin renunciar al control cercano al hardware que había hecho popular a su lenguaje base [1, 2].



La década de 1990 trajo los lenguajes que hoy dominan la programación de alto nivel. Guido van Rossum publicó Python en 1991, con una sintaxis pensada para leerse casi como pseudocódigo [1]. En 1995 —“un año especialmente destacable”, según lo describe Pérez López [1]— James Gosling presentó Java en Sun Microsystems: un lenguaje que compila a un código intermedio (bytecode) ejecutado por una máquina virtual, lo que le permite correr sin cambios en cualquier plataforma que tenga esa máquina virtual instalada [1, 2]. El mismo año aparecieron JavaScript, PHP y Ruby, cada uno resolviendo una necesidad distinta de la naciente World Wide Web [1]. En el año 2000, Microsoft presentó C#, un lenguaje orientado a objetos diseñado junto con la plataforma .NET como respuesta directa a Java [1]. Desde entonces han surgido lenguajes como Go, Swift, Kotlin o Rust, cada uno enfocado en un problema puntual —conurrencia, seguridad de memoria, desarrollo móvil— que los lenguajes anteriores dejaban sin resolver del todo [1].

Clasificación por nivel de abstracción

Los lenguajes de programación pueden ordenarse en una escalera de cuatro niveles, según qué tan cerca estén del hardware o del pensamiento humano.

En la base está el hardware propiamente dicho: voltaje encendido o apagado, representado como ceros y unos, que es lo único que un procesador entiende de forma nativa. Justo encima se ubican los lenguajes de **bajo nivel**, principalmente el ensamblador, cuyas instrucciones mnemónicas se traducen de forma directa casi una a una a instrucciones de la arquitectura de la CPU [1, 2]. Estos lenguajes ofrecen el máximo control sobre la máquina, pero sus programas son difíciles de escribir, depurar y trasladar de un equipo a otro [2].

Entre el bajo y el alto nivel se sitúan los lenguajes de **nivel medio**, cuyo ejemplo más citado es precisamente C: una sintaxis suficientemente estructurada como para programar con comodidad, pero con acceso directo a punteros y a la memoria del sistema [1]. C++ hereda esa misma posición intermedia, al



añadir sobre C las herramientas de la programación orientada a objetos sin perder ese acceso de bajo nivel [1, 2].

Finalmente, los lenguajes de **alto nivel**, Java, Python, JavaScript, C#, entre muchos otros—priorizan una sintaxis cercana al razonamiento humano y delegan en el propio lenguaje tareas como la gestión de memoria. Esto los hace más fáciles de escribir, depurar, mantener y trasladar entre plataformas, a costa de un menor control directo sobre los recursos de la máquina [2].

Comparación entre un lenguaje de nivel medio y uno de alto nivel: C++ y Java

Característica	Java	C++
Traducción	Compila a bytecode; lo ejecuta la JVM	Compila directo a código máquina nativo
Gestión de memoria	Recolector de basura automático	El programador la administra (punteros)
Nivel de abstracción	Alto nivel	Nivel medio
IDE / entorno típico	IntelliJ IDEA, Eclipse, NetBeans	Visual Studio, Code::Blocks, VS Code
Prioriza	Portabilidad y seguridad	Rendimiento y control del hardware
Creador / año	James Gosling, 1995 (Sun Microsystems)	Bjarne Stroustrup, 1985 (Bell Labs)

Tabla 1. Comparación entre C++ y Java.

Aunque ambos lenguajes provienen de la misma familia Java toma buena parte de su sintaxis de C y C++ [2], sus decisiones de diseño responden a prioridades distintas. C++ compila directamente a código máquina nativo, lo que le da un rendimiento cercano al del hardware, pero obliga al programador a administrar la memoria manualmente mediante punteros. Java, en cambio, compila a un bytecode intermedio que ejecuta la máquina virtual de Java (JVM); esa capa adicional tiene un costo de



rendimiento, pero a cambio libera al programador de gestionar la memoria de eso se encarga el recolector de basura y permite que el mismo programa corra sin cambios en cualquier sistema que tenga instalada la JVM.

Esa diferencia de filosofía se refleja también en las herramientas que suele usar cada comunidad. Para C++, los entornos más habituales son Visual Studio y Code::Blocks, o editores extensibles como Visual Studio Code; para Java, IDEs pensados específicamente para el lenguaje, como IntelliJ IDEA, Eclipse o NetBeans. La elección suele depender de la plataforma de destino del proyecto: C++ se mantiene como referencia en sistemas donde el rendimiento es crítico videojuegos, sistemas embebidos, mientras que Java sigue siendo una opción sólida para aplicaciones empresariales que deben ejecutarse en distintos sistemas operativos sin recompilarse.

La Conexión entre C++ y Java

El estudio conjunto de C++ y Java proporciona una base estructural indispensable para comprender a fondo la arquitectura del software moderno. Ambos lenguajes comparten una sintaxis derivada de C y se fundamentan en el paradigma de la programación orientada a objetos (POO), lo que permite que los conceptos de clases, herencia y polimorfismo fluyan de manera natural entre ambos. La razón de enseñarlos en conjunto radica en su complementariedad: mientras que C++ ofrece un acercamiento de bajo nivel que exige gestionar directamente la memoria y comprender cómo interactúa el código con el hardware, Java abstrae estas complejidades mediante su máquina virtual (JVM) y el recolector de basura. Esta transición pedagógica permite que el estudiante desarrolle un criterio técnico integral, entendiendo primero qué ocurre en las entrañas del sistema con C++, para luego aplicar esos mismos principios lógicos en el desarrollo de aplicaciones multiplataforma y empresariales más seguras y escalables con Java.

Visual Studio Code: Versatilidad y Adaptabilidad en el Desarrollo Moderno



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: JULIO – DICIEMBRE 2026



A diferencia de los entornos de desarrollo integrados (IDE) tradicionales como NetBeans o Eclipse que suelen ser pesados y estar orientados a un lenguaje específico, la exigencia de utilizar Visual Studio Code responde a su naturaleza como una herramienta ligera, altamente personalizable y que representa el estándar actual en la industria. Su mayor fortaleza es su extrema versatilidad, impulsada por un ecosistema de extensiones inagotable que permite compilar, ejecutar y depurar proyectos de C++, Java e incluso conectar bases de datos desde una única interfaz. Al no ser un IDE "caja negra" que hace todo automáticamente, VS Code fuerza al desarrollador a comprender los procesos subyacentes: desde la configuración de variables de entorno y archivos JSON, hasta el dominio de la terminal integrada y el control de versiones. Esta adaptabilidad no solo optimiza los recursos de la computadora, sino que entrena la capacidad de configurar entornos de trabajo profesionales a medida, independientemente de la tecnología que el proyecto demande.